

Activitat 3: Aplicació de Böttcher

Sistemes Dinàmics - 2024/2025

Max Genestar i Pau Ortega

Contents

1	Introducció	2
2	Coeficients b_n	3
3	Conjugació via $\tau(z)$	5
4	Estudi F_c	6
5	Jocs de Prova	7
6	Conjunts de Fatou i Julia de $Q_C(z) = z^2 + c$	15
7	Visualització de conjunts conjugats via $\tau \circ \varphi_c$	20
7.1	Circumferències a la conca d'atracció de l'infinit	21
7.2	Línies cap a l'infinit	26
8	Annex	29

1 Introducció

En aquest treball s'estudia el comportament local de certes funcions holomorfes a l'entorn de punts fixos atractius, mitjançant l'ús de la **conjugació de Böttcher**. Aquesta tècnica, permet linealitzar una funció en un entorn, transformant-la en una funció més senzilla amb el mateix comportament dinàmic local.

La idea original d'aquesta conjugació es remunta a **Lucjan Emil Böttcher** (1872–1937), un matemàtic polonès que va estudiar la iteració de funcions racionals molt abans que aquest camp es consolidés com a branca formal de les matemàtiques. Malgrat que els seus treballs no van rebre gaire reconeixement en el seu temps, avui dia se'l considera una persona molt important per l'estudi de dinàmica complexa.

En aquest projecte ens centrem en la funció racional $F_c(z) = \frac{z^2}{1+z^2c}$, que és conjugada mitjançant una transformació a la funció quadràtica $Q_c(z) = z^2 + c$. A partir del desenvolupament de Taylor de F_c al voltant de l'origen, es construeix iterativament l'aplicació de Böttcher $\varphi(z)$, trobant els coeficients b_n que satisfan la relació funcional $F_c(\varphi(z)) = \varphi(z^2)$. A més a més, es representen gràficament els conjunts de Julia i de Fatou associats per diferents valors del paràmetre c .

2 Coeficients b_n

Per calcular els coeficients b_n de l'aplicació de Böttcher conjuga localment amb $f(z)$ i la funció z^2 en un entorn de l'origen, és a dir, es verifica que $f(\varphi(z)) = \varphi(z^2)$ on:

Començem calculant $f(\varphi(z))$. Per fer-ho substituïm la forma general de la funció de Böttcher $\varphi(z) = b_1z + b_2z^2 + b_3z^3 + \dots$ en l'expressió de la funció $f(z)$, que té un desenvolupament de Taylor començant a partir de a_2z^2 :

$$f(\varphi(z)) = a_2\varphi(z)^2 + a_3\varphi(z)^3 + a_4\varphi(z)^4 + \dots$$

Utilitzant la forma de $\varphi(z)$, podem expandir els termes com a sèries de potències de z .

Següidament calculem l'altra igualtat de l'expressió. La funció $\varphi(z^2)$ es pot escriure substituint z^2 en la fórmula de $\varphi(z)$:

$$\varphi(z^2) = b_1z^2 + b_2z^4 + b_3z^6 + \dots$$

Un cop tenim calculades les dues expressions podem igualar els termes que tinguin les mateixes potències de z per així trobar una relació dels coeficients de b_n .

En el cas que se'ns presenta en aquesta pràctica hem vist que $f(0) = f'(0) = 0$, per tant, a l'hora d'igualar coeficients per trobar els coeficients de la conjugació començarem a partir del grau 2.

$$\textbf{Grau 2: } a_2b_1^2 = b_1 \rightarrow b_1 = \frac{1}{a_2}$$

$$\textbf{Grau 3: } 2a_2b_1b_2 + a_3b_1^3 = 0 \rightarrow b_2 = -\frac{a_3b_1^3}{2a_2} = -\frac{a_3}{2a_2^3}$$

$$\textbf{Grau 4: } a_2(2b_1b_3 + b_2^2) + a_3(3b_1^2b_2) + a_4b_1^4 = b_2 \rightarrow b_3 = \frac{b_2 - a_2b_2^2 - 3a_3b_1^2b_2 - a_4b_1^4}{2}$$

$\vdots$

Es pot observar que obtindrem dues expressions. Una per als coeficients senars i una altra per als parells. A més, si ens hi fixem bé, podem reutilitzar l'expressió d' H_n que vam fer servir a la pràctica 1, adaptant-la lleugerament per al cas actual.

Termes parells z^{2k} :

$$a_2(2b_1b_{2k-1} + 2b_2b_{2k-2} + \dots + 2b_{k-1}b_{k+1} + b_k^2) + H_{2k} = b_k$$

$$b_{2k-1} = \frac{b_k - H_{2k} - a_2(\sum_{l=2}^{k-1} 2b_lb_{2k-l} + b_k^2)}{2}$$

Termes senars z^{2k+1} :

$$a_2(2b_1b_{2k} + 2b_2b_{2k-1} + \cdots + 2b_kb_{k+1}) + H_{2k+1} = 0$$

$$b_{2k} = -\frac{H_{2k+1} + a_2 \sum_{l=1}^{k-1} 2b_{l+1}b_{2k-l}}{2}$$

H_n :

Recordant l'expressió de la Pràctica 1, teníem que:

$$H_n(a_2, \dots, a_{n-1}, b_1, \dots, b_{n-2}) = \sum_{j=2}^n a_j \left(\sum_{k=1}^{n-2} b_k x^k \right)^j$$

En aquesta pràctica, cal fer unes petites modificacions:

- Comencem a partir del terme a_3 , per tant la suma externa s'inicia amb $j = 3$ en lloc de $j = 2$.
- El segon sumatori el fem fins a $n - 2$.

3 Conjugació via $\tau(z)$

Considerem la funció quadràtica $Q_c(z) = z^2 + c$, on $c \in \mathbb{C}$. En aquest apartat, comprovarem que Q_c és conjugada amb la funció $F_c(z) = \frac{z^2}{1+z^2c}$ mitjançant l'aplicació $\tau(z) = \frac{1}{z}$.

$$\begin{array}{ccc}
 \mathbb{C} & \xrightarrow{F_c(z)} & \mathbb{C} \\
 \tau(z) \downarrow & \equiv & \downarrow \tau(z) \\
 \mathbb{C} & \xrightarrow{Q_c(z)} & \mathbb{C}
 \end{array}$$

Per comprovar que les funcions $Q_c(z)$ i $F_c(z)$ estan relacionades mitjançant la conjugació a través de l'aplicació $\tau(z)$, cal demostrar que es compleix la següent identitat:

$$Q_c \circ \tau = \tau \circ F_c$$

Comprovem-ho pas a pas:

$$Q_c(\tau) = \tau^2 + c = \frac{1}{z^2} + c$$

$$\tau(F_c) = \frac{1}{(F_c)} = \frac{(1 + cz^2)}{z^2} = \frac{1}{z^2} + c$$

i per tant, les funcions Q_c i F_c són conjugades mitjançant τ .

Fem aquesta conjugació per analitzar el comportament de $Q_c(z)$ a l'infinit que ens resulta difícil d'estudiar directament. Com que $\tau(z) = \frac{1}{z}$ envia ∞ a 0 i les dues funcions conjuguen localment, la dinàmica de Q_c prop de ∞ es transfereix a la dinàmica de F_c al voltant de 0.

4 Estudi F_c

La funció que volem estudiar és:

$$F_c(z) = \frac{z^2}{1 + z^2 c}$$

On c és un paràmetre.

Si substituïm $z = 0$ en $F_c(z)$ es pot veure fàcilment que $F_c(0) = 0$.

Ara comprovarem que $F'_c(0)$ també és 0, per fer-ho utilitzarem la regla del quocient i evaluarem en $z = 0$.

$$F'_c(z) = \frac{(2z)(1 + z^2 c) - z^2(2zc)}{(1 + z^2 c)^2} = \frac{2z}{(1 + z^2 c)^2}$$

Per $z = 0$:

$$F'_c(0) = \frac{2(0)}{(1 + 0^2 c)^2} = \frac{0}{1} = 0$$

Finalment anem a veure que $F''_c(0) = 1$.

$$F''_c(z) = \frac{(2)(1 + z^2 c)^2 - 2z(4zc + 4z^3 c^2)}{(1 + z^2 c)^4}$$

Substituïm $z = 0$:

$$F''_c(0) = \frac{(2)(1 + 0^2 c)^2 - 2(0)(4(0)c + 4(0)^3 c^2)}{(1 + (0)^2 c)^4} = \frac{2}{1} = 2$$

El desenvolupament de Taylor de la funció $F_c(z)$ al voltant de l'origen és de la forma:

$$F_c(z) = F_c(0) + F'_c(0)z + \frac{F''_c(0)}{2!}z^2 + \frac{F'''_c(0)}{3!}z^3 + \dots$$

Substituïm els valors trobats en el desenvolupament de Taylor:

$$F_c(z) = 0 + 0 \cdot z + \frac{2}{2!}z^2 + \mathcal{O}(z^3)$$

$$F_c(z) = z^2 + \mathcal{O}(z^3)$$

Tot i que aquest resultat és força interessant, el que ens pot resultar més útil a l'hora de realitzar la pràctica és obtenir una expressió del Taylor de $F_c(z)$, que mitjançant una eina tan útil com la sèrie geomètrica que ens dona el següent resultat i permet calcular el radi de convergència de la sèrie:

$$F_c(z) = \frac{z^2}{1 + cz^2} = z^2 \frac{1}{1 - (-cz^2)} = z^2 \sum_{n=0}^{\infty} (-c)^n z^{2n} = \sum_{n=0}^{\infty} (-c)^n z^{2n+2}, \quad |cz^2| < 1$$

5 Jocs de Prova

En aquest apartat, determinarem numèricament l'aplicació de Böttcher ϕ_c per a diversos valors de c i analitzarem els errors que tenen els coeficients a mesura que els anem calculant amb gràfics.

Primer de tot estudiarem una mica el comportament de $F_c(z)$, primer hem de calcular els seus punts fixos. Un punt fix de la funció $F_c(z)$ és aquell valor de z per al qual es compleix:

$$F_c(z) = z$$

Tenim que $F_c(z) = \frac{z^2}{1+z^2c}$. Per tant, volem resoldre l'equació:

$$\frac{z^2}{1+z^2c} = z$$

Reorganitzant tots els termes a un costat de l'equació:

$$z^3c - z + z^2 = 0$$

Aquesta és una equació de grau 3, per tant té tres solucions (segons el Teorema Fonamental de l'Àlgebra). Les solucions es poden descompondre de la següent manera:

- **Solució trivial:** Una solució evident és:

$$z = 0$$

- **Equació quadràtica:** Per a les altres solucions, considerem l'equació quadràtica resultant:

$$z^2c - z + 1 = 0$$

La solució d'aquesta equació quadràtica és:

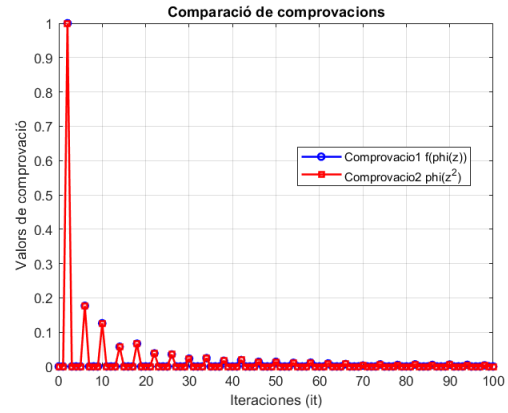
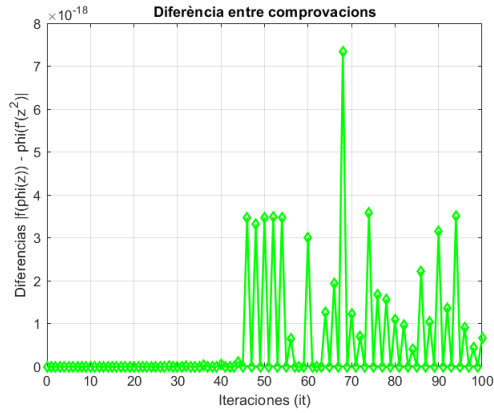
$$z = \frac{-(-1) \pm \sqrt{(-1)^2 - 4(c)(1)}}{2(c)} = \frac{1 \pm \sqrt{1 - 4c}}{2c}$$

Aquesta solució dependrà del valor de c :

- Si $1 - 4c > 0$, hi haurà dues solucions reals diferents.
- Si $1 - 4c = 0$, hi haurà una solució real doble.
- Si $1 - 4c < 0$, hi haurà dues solucions complexes conjugades.

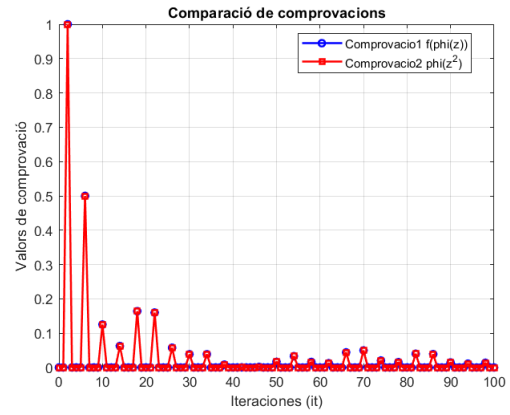
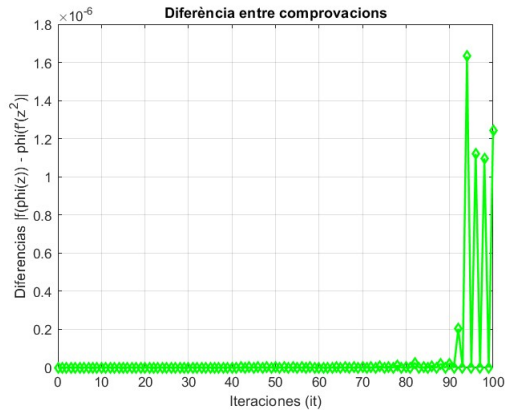
$$c = 0.25 + 0.25i$$

n	b_n	dif(n)
0	0.000000000000e+00 + 0.000000000000i	0.0000e+00
1	1.000000000000 + 0.000000000000i	0.0000e+00
2	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
3	0.125000000000 + 0.125000000000i	0.0000e+00
4	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
5	0.062500000000 + 0.109375000000i	0.0000e+00
6	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
7	-0.009765625000 + 0.056640625000i	0.0000e+00
8	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
9	0.012329101562 + 0.065429687500i	0.0000e+00
10	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
11	-0.019760131836 + 0.032730102539i	0.0000e+00
12	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
13	-0.021629333496 + 0.027673721313i	0.0000e+00
14	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
15	-0.022061109543 + 0.005892515182i	0.0000e+00
16	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
17	-0.002622939646 + 0.024163365364i	0.0000e+00
18	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
19	-0.014294003136 + 0.008603270166i	0.0000e+00
20	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
21	-0.015942005906 + 0.009779057582i	0.0000e+00
22	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
23	-0.013410464671 + -0.002376900127i	0.0000e+00
24	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
25	-0.012553999039 + 0.005281682686i	0.0000e+00



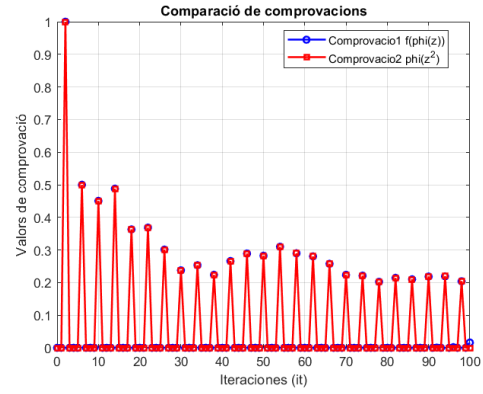
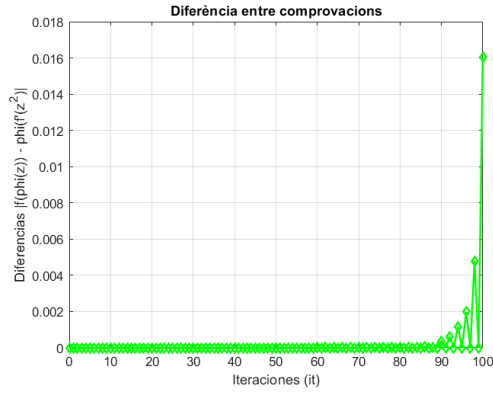
$$c = -1$$

n	b_n	dif(n)
0	0.000000000000e+00 + 0.000000000000i	0.0000e+00
1	1.000000000000 + 0.000000000000i	0.0000e+00
2	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
3	-0.500000000000 + 0.000000000000i	0.0000e+00
4	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
5	0.125000000000 + 0.000000000000i	0.0000e+00
6	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
7	0.062500000000 + 0.000000000000i	0.0000e+00
8	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
9	-0.164062500000 + 0.000000000000i	0.0000e+00
10	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
11	0.160156250000 + 0.000000000000i	0.0000e+00
12	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
13	-0.057617187500 + 0.000000000000i	0.0000e+00
14	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
15	-0.038574218750 + 0.000000000000i	0.0000e+00
16	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
17	0.038665771484 + 0.000000000000i	0.0000e+00
18	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
19	-0.008468627930 + 0.000000000000i	0.0000e+00
20	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
21	3.623962402344e-04 + 0.000000000000i	0.0000e+00
22	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
23	0.001722335815 + 0.000000000000i	0.0000e+00
24	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
25	0.016601324081 + 0.000000000000i	0.0000e+00



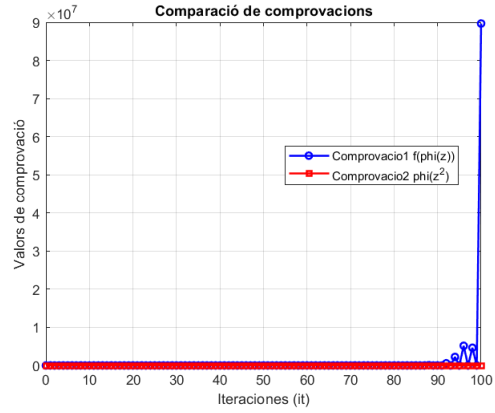
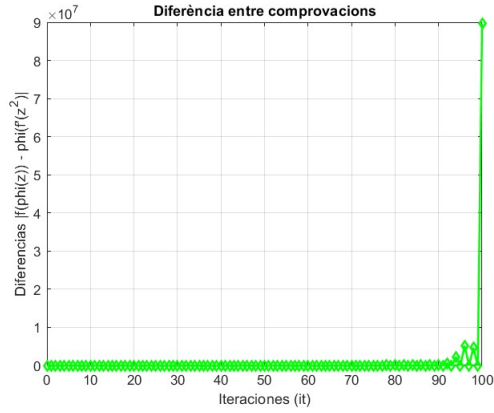
$$c = i$$

n	b_n	dif(n)
0	0.000000000000e+00 + 0.000000000000i	0.0000e+00
1	1.000000000000 + 0.000000000000i	0.0000e+00
2	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
3	0.000000000000 + 0.500000000000i	0.0000e+00
4	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
5	-0.375000000000 + 0.250000000000i	0.0000e+00
6	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
7	-0.375000000000 + -0.312500000000i	0.0000e+00
8	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
9	0.117187500000 + -0.343750000000i	0.0000e+00
10	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
11	0.359375000000 + -0.082031250000i	0.0000e+00
12	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
13	0.145507812500 + 0.263671875000i	0.0000e+00
14	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
15	-0.102539062500 + 0.214355468750i	0.0000e+00
16	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
17	-0.251129150391 + 0.032470703125i	0.0000e+00
18	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
19	-0.135375976562 + -0.178421020508i	0.0000e+00
20	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
21	0.077739715576 + -0.254554748535i	0.0000e+00
22	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
23	0.287441253662 + -0.031423568726i	0.0000e+00
24	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
25	0.155661344528 + 0.235613822937i	0.0000e+00



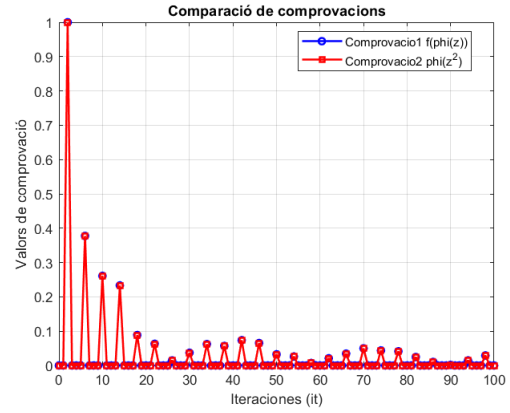
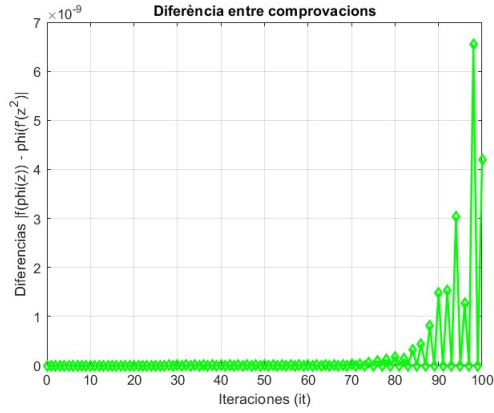
$$c = -1.7549$$

n	b_n	dif(n)
0	0.000000000000e+00 + 0.000000000000i	0.0000e+00
1	1.000000000000 + 0.000000000000i	0.0000e+00
2	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
3	-0.877450000000 + 0.000000000000i	0.0000e+00
4	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
5	0.716152753750 + 0.000000000000i	0.0000e+00
6	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
7	-0.534034721297 + 0.000000000000i	0.0000e+00
8	-0.000000000000e+00 + 0.000000000000i	8.8818e-16
9	0.321856291144 + 0.000000000000i	0.0000e+00
10	-0.000000000000e+00 + 0.000000000000i	1.4433e-15
11	-0.105165978091 + 0.000000000000i	0.0000e+00
12	-0.000000000000e+00 + 0.000000000000i	3.5527e-15
13	-0.068249156411 + 0.000000000000i	0.0000e+00
14	-0.000000000000e+00 + 0.000000000000i	1.1546e-14
15	0.187165445551 + 0.000000000000i	0.0000e+00
16	-0.000000000000e+00 + 0.000000000000i	1.3500e-13
17	-0.280807432428 + 0.000000000000i	0.0000e+00
18	-0.000000000000e+00 + 0.000000000000i	2.5757e-14
19	0.339611967462 + 0.000000000000i	0.0000e+00
20	-0.000000000000e+00 + 0.000000000000i	1.4211e-13
21	-0.331530810951 + 0.000000000000i	0.0000e+00
22	-0.000000000000e+00 + 0.000000000000i	9.4125e-13
23	0.268473463298 + 0.000000000000i	0.0000e+00
24	-0.000000000000e+00 + 0.000000000000i	1.2506e-12
25	-0.176056087819 + 0.000000000000i	0.0000e+00



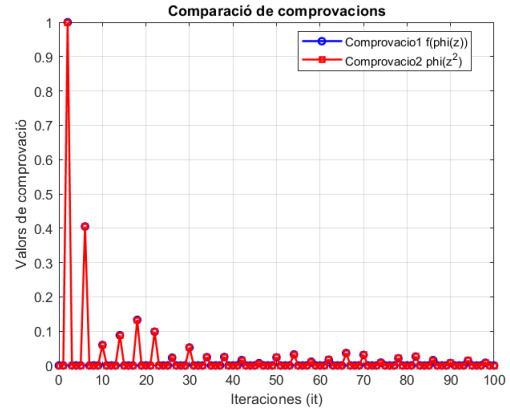
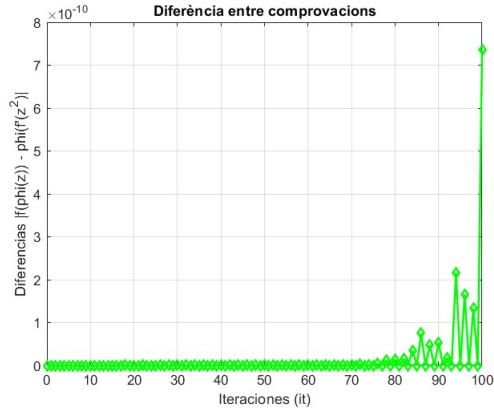
$$c = -0.12256 + 0.74486i$$

n	b_n	dif(n)
0	0.000000000000e+00 + 0.000000000000i	0.0000e+00
1	1.000000000000 + 0.000000000000i	0.0000e+00
2	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
3	-0.061280000000 + 0.372430000000i	0.0000e+00
4	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
5	-0.233063299750 + 0.117747468800i	0.0000e+00
6	-0.000000000000e+00 + 0.000000000000i	5.5511e-17
7	-0.139250195661 + -0.187122229890i	0.0000e+00
8	-0.000000000000e+00 + 0.000000000000i	6.2063e-17
9	0.065655427397 + -0.059504927625i	0.0000e+00
10	-0.000000000000e+00 + 0.000000000000i	9.3095e-17
11	0.062323159883 + -0.009670027158i	0.0000e+00
12	-0.000000000000e+00 + 0.000000000000i	6.2063e-17
13	-0.010285529758 + 0.010689611406i	0.0000e+00
14	-0.000000000000e+00 + 0.000000000000i	3.1032e-16
15	0.037015809186 + -0.001106076371i	0.0000e+00
16	-0.000000000000e+00 + 0.000000000000i	2.2420e-16
17	-0.010984876945 + 0.061267245172i	0.0000e+00
18	-0.000000000000e+00 + 0.000000000000i	5.0287e-16
19	-0.056417230746 + 0.008045026704i	0.0000e+00
20	-0.000000000000e+00 + 0.000000000000i	4.7123e-16
21	-0.023674018505 + -0.069830379627i	0.0000e+00
22	-0.000000000000e+00 + 0.000000000000i	8.4953e-16
23	0.060569297586 + -0.024604580312i	0.0000e+00
24	-0.000000000000e+00 + 0.000000000000i	5.2135e-16
25	0.019462187481 + 0.026046810833i	0.0000e+00



$$c = -0.796145 + 0.148172i$$

n	b_n	dif(n)
0	0.000000000000e+00 + 0.000000000000i	0.0000e+00
1	1.000000000000 + 0.000000000000i	0.0000e+00
2	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
3	-0.398072500000 + 0.074086000000i	0.0000e+00
4	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
5	0.030423219790 + -0.051431797705i	0.0000e+00
6	-0.000000000000e+00 + 0.000000000000i	5.7220e-17
7	0.088148201581 + -0.001442934058i	0.0000e+00
8	-0.000000000000e+00 + 0.000000000000i	0.0000e+00
9	-0.128719412364 + 0.033254984650i	0.0000e+00
10	-0.000000000000e+00 + 0.000000000000i	7.8505e-17
11	0.088745235838 + -0.042560746150i	0.0000e+00
12	-0.000000000000e+00 + 0.000000000000i	1.7554e-16
13	0.003546624104 + 0.022448591259i	0.0000e+00
14	-0.000000000000e+00 + 0.000000000000i	5.0037e-17
15	-0.051454474512 + 0.009442059716i	0.0000e+00
16	-0.000000000000e+00 + 0.000000000000i	1.7554e-16
17	0.010525973192 + -0.021732861957i	0.0000e+00
18	-0.000000000000e+00 + 0.000000000000i	2.9966e-16
19	0.022806680055 + 0.009454410241i	0.0000e+00
20	-0.000000000000e+00 + 0.000000000000i	1.8768e-16
21	-0.015394733163 + 0.003025564473i	0.0000e+00
22	-0.000000000000e+00 + 0.000000000000i	3.6619e-16
23	0.001721352134 + -0.006581953306i	0.0000e+00
24	-0.000000000000e+00 + 0.000000000000i	1.4305e-16
25	0.023399705227 + 0.000946354993i	0.0000e+00



Els resultats obtinguts mostren que, per valors moderats de c , l'aplicació de Böttcher es pot calcular amb bona precisió numèrica, cosa que indica una convergència estable de la sèrie.

Tanmateix, a mesura que augmenta l'índex n , el càlcul dels coeficients esdevé més sensible, fet que provoca un creixement de l'error, especialment en zones properes al límit de convergència de la sèrie.

La validesa dels resultats s'ha verificat utilitzant que $f(\varphi(z)) = \varphi(z^2)$.

6 Conjunts de Fatou i Julia de $Q_C(z) = z^2 + c$

Explicació dels Codis

En aquest apartat analitzarem els conjunts de Fatou i Julia que obtenim pels diferents valors de c que hem proposat i podem substituir a la funció quadràtica $Q_c(z) = z^2 + c$. Segons el valor de c que posem a la funció, podrem veure en aquesta secció com la forma d'aquests conjunts canvia enormement.

Abans d'entrar a veure quina forma tenen els conjunts en la funció quadràtica pels diferents valors de c que considerem, parlarem dels codis que hem fet servir per obtenir la representació d'aquests conjunts. Els codis en essència són els mateixos que havíem desenvolupat en la pràctica 2 però a l'hora d'obtenir el conjunt de Fatou ple pel cas de $c = i$ ens hem trobat amb problemes a l'hora de fer la representació donat que la imatge que obteníem era com si tots els punts del requadre del pla complex que representàvem es trobessin al conjunt de Fatou.

Llavors ara passarem a exposar els dos codis que hem fet servir per obtenir les imatges del conjunt ple de Fatou, i quan exposem les imatges de la funció quadràtica, per les diferents c , dels conjunts de Julia i Fatou, exposarem les imatges del conjunt ple de Fatou obtingudes amb els dos codis.

```
1 %Versio 1
2 BM=ones(size(Z)); %Matriu Binaria (Binary Matrix)
3
4 %Avaluacio dels punts [-2,2]x[-2,2] en Q_c
5 for i=1:N
6     Z1=Q_c(Z1);
7 end
8
9 %Modifiquem BM segons la forma dels punts despres de les iteracions
10 for i=1:size(Z1,1)
11     for j=1:size(Z1,2)
12         if isnan(abs(Z1(i,j))) || abs(Z1(i,j))>R
13             BM(i,j)=0;
14         end
15     end
16 end
```

En aquest codi, que ja vam fer servir a la pràctica 2, mitjançant una matriu que conté diferents punts del conjunt $[-2, 2] \times [-2, 2]$ i d'aquesta matriu es genera una altra de les mateixes dimensions però tot amb uns. Un cop tenim això, mirem si el mòdul dels punts de la matriu, quan els hi hem aplicat una certa quantitat de vegades la funció $Q_c(z)$, compleix la condició de ser o bé NaN o ser superior al radi d'escapament, els punts amb un mòdul que compleixi aquestes condicions són marcats com a 0.

La segona versió del codi que hem hagut de desenvolupar és igual d'eficaç i eficient que la primera i ens soluciona el problema que teníem amb el cas $c = i$.


```

1 %Versió 2
2
3 BM=zeros(size(N)); %Matriu de zeros
4
5 for n = 1:N
6     Z1 = Q_c(Z1); %Avaluem els punts de [-2,2]x[-2,2] en Q_c
7     BM = BM + (abs(Z1) <= R); % Suma 1 només als punts que no escapen
8 end

```

En aquesta versió del codi realitzem el mateix procediment que amb la primera però de manera que el cas més subtil, $c = i$, es pugui representar correctament. Les diferències principals serien que definim una matriu de zeros a la qual en lloc de canviar binàriament els seus valors, els afegim un 1 en cas que els punts no escapin cap a l'infinit. Pels diferents valors de c que hem trobat les imatges que obtenim són força similars pels dos codis, però la imatge resultant en el cas $c = i$ resulta molt més il·lustrativa si fem servir el segon codi.

A més d'aquests dos codis, també hem fet certes modificacions al codi que vam fer servir en la pràctica 2 per representar el conjunt de Julia. Els canvis que hem fet han estat majorment en l'equació que havíem de resoldre per trobar les preimatges del punt fix repulsor amb què treballem, i ens han donat el següent codi:

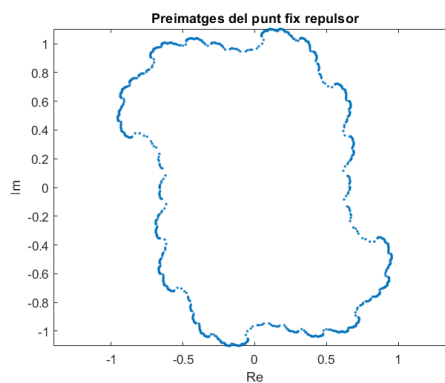
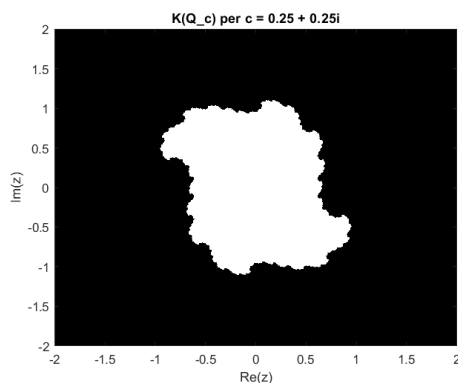
```

1 N=10; % Quantitat de iteracions
2 preimatges=z1;
3
4 for i=1:N
5     newpre=[];
6     for j=1:length(preimatges)
7         w = preimatges(j);
8         % Les preimatges son les solucions de z^2-w+c=0
9         a = 1;
10        b = 0;
11        d = c-w;
12        arrel2 = b^2 - 4*a*d;
13        x1 = (-b + sqrt(arrel2)) / (2*a);
14        x2 = (-b - sqrt(arrel2)) / (2*a);
15        newpre = [newpre, x1, x2];
16    end
17    % Afegim les noves preimatges al vector de preimatges
18    preimatges = [preimatges, newpre];
19 end

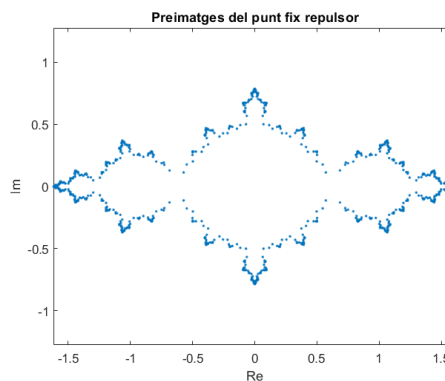
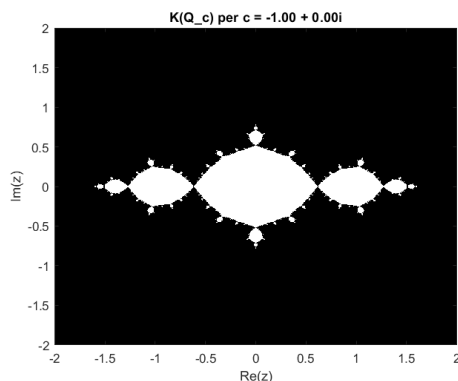
```

Abans de veure les representacions dels conjunts de Julia i ple de Fatou per Q_c és important que esmentem que tots dos punts fixos d'aquesta funció, pels casos que hem estudiat, són repulsors, només hi ha una excepció que és pel cas de $c = 0.25 + 0.25i$.

$$c = 0.25 + 0.25i$$



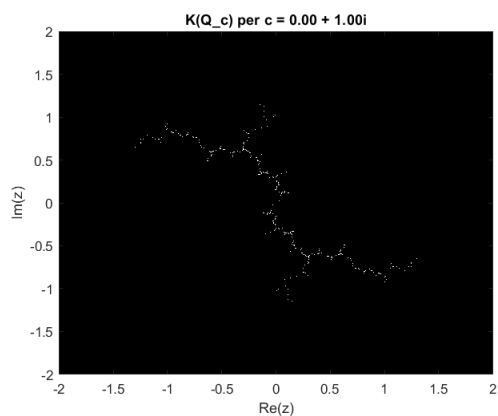
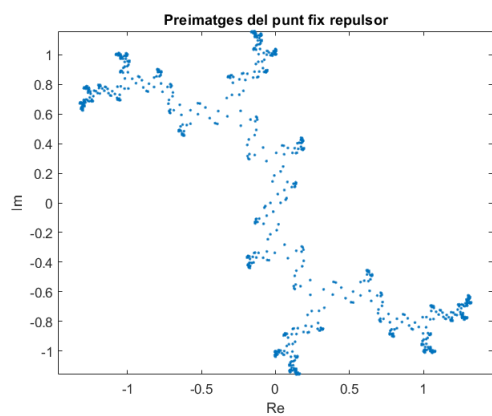
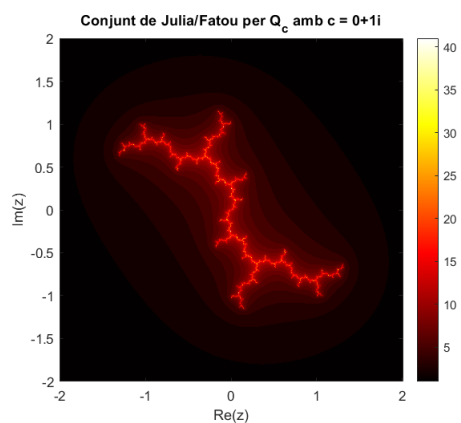
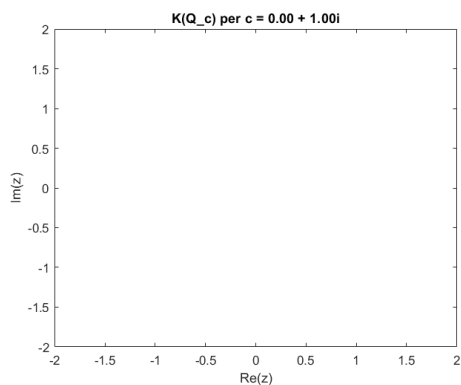
$$c = -1$$



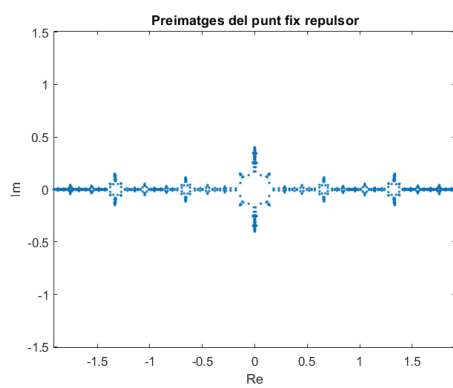
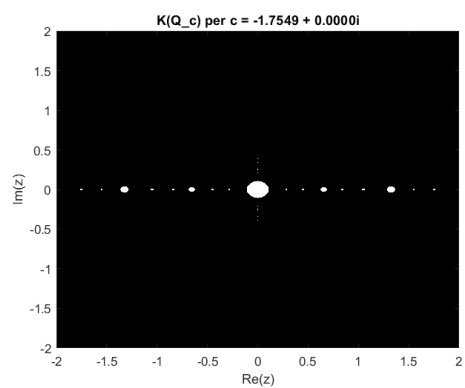
$$c = i$$

En aquestes dues primeres imatges es pot veure l'error que ens va fer haver de crear una altra versió del codi que fèiem servir inicialment per obtenir la representació del conjunt ple de Fatou, com podem veure a la primera imatge de la primera fila s'està representant tot el pla $[-2, 2] \times [-2, 2]$ com si fos el conjunt ple de Fatou, la qual cosa és errònia. A la segona imatge de la mateixa fila es pot veure una representació més acurada amb un mapa de colors on els colors més càlids són els que escapen més ràpidament i els colors més apagats el contrari.

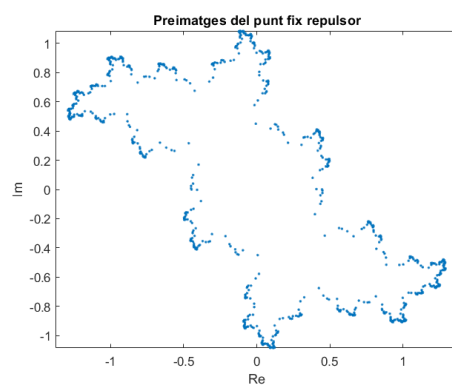
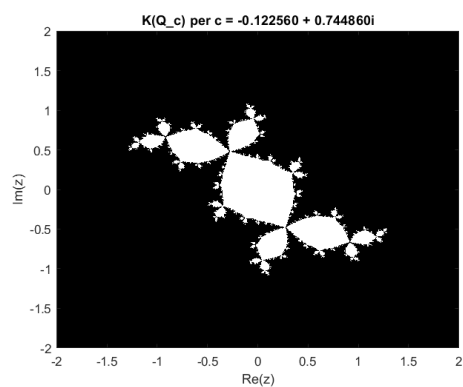
Una altra solució que hem trobat per representar el conjunt d'òrbites acotades és reduir la quantitat d'iteracions que fem en la versió original del codi, d'aquesta manera evitem tenir problemes al tractar amb l'extrema precisió que requereix aquest cas. Tot i que si això no es feia correctament, també obteníem una imatge a la qual es representava tot el pla anteriorment esmentat com la conca d'atracció d'infiní.



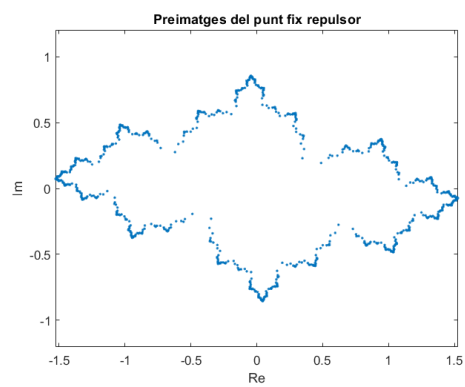
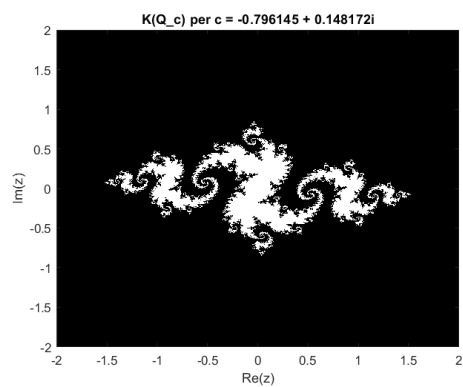
$$c = -1.7549$$



$$c = -0.12256 + 0.74486i$$



$$c = -0.796145 + 0.148172i$$



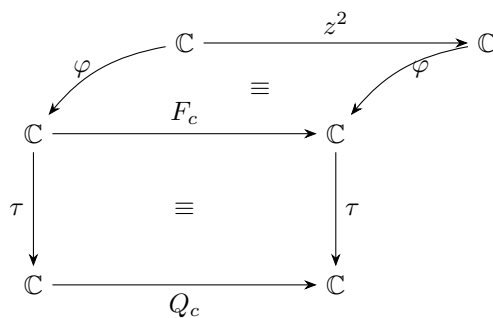
7 Visualització de conjunts conjugats via $\tau \circ \varphi_c$

En aquesta secció estudiarem la composició $\tau \circ \varphi_c$ sota certs conjunts.

Per a cada $k = 1, \dots, 15$, definim $s_k = 0.05 \cdot k$, i considerem:

$$\{s_k e^{2\pi i \theta} \mid 0 \leq \theta \leq 1\} \quad \text{i} \quad \{r e^{2\pi i s_k} \mid 0 \leq r \leq 1\}$$

Un cop definit el conjunt anterior ara podem pensar en el següent mapa de funcions i conjugacions amb les què hem treballat durant la realització de la pràctica:



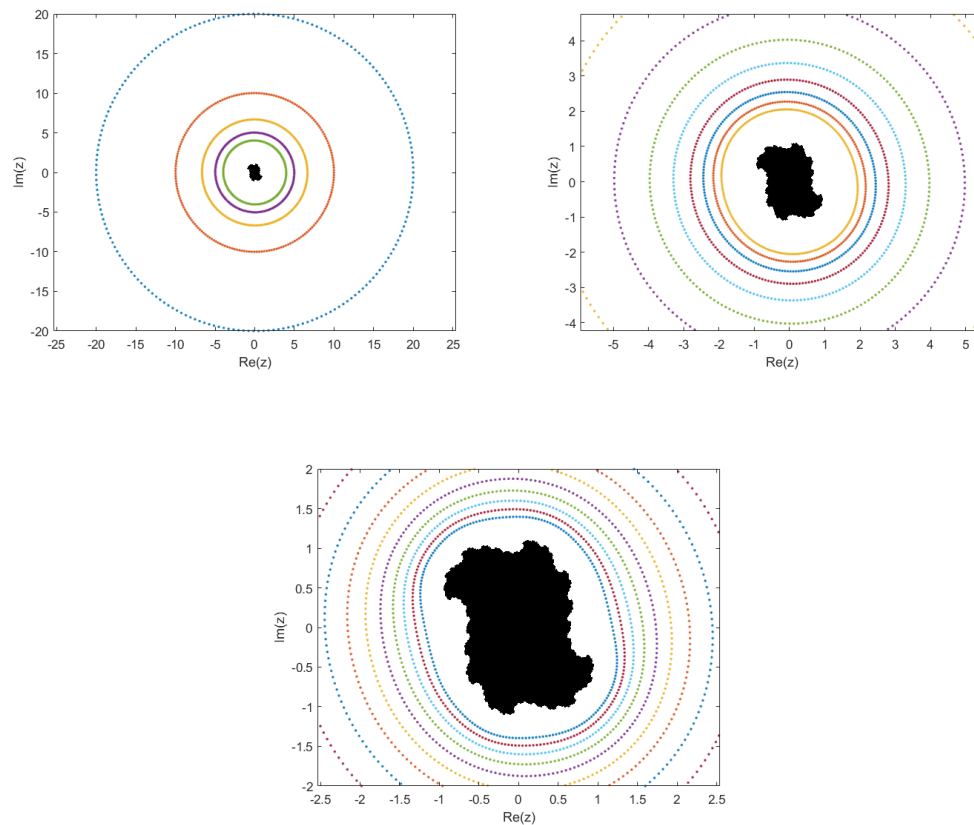
En aquest graf es representen les diferents funcions amb les quals hem treballat a la pràctica juntament amb les conjugacions que existeixen entre elles. Gràcies a aquesta representació podem analitzar i explicar què és el que està passant quan componem τ amb φ .

7.1 Circumferències a la conca d'atracció de l'infinit

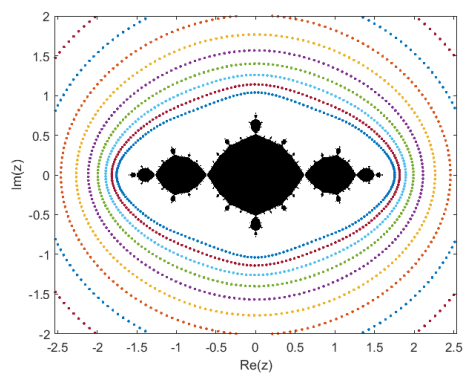
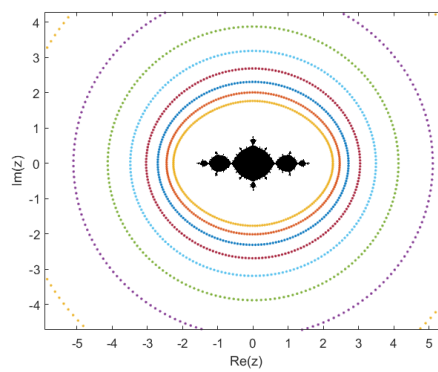
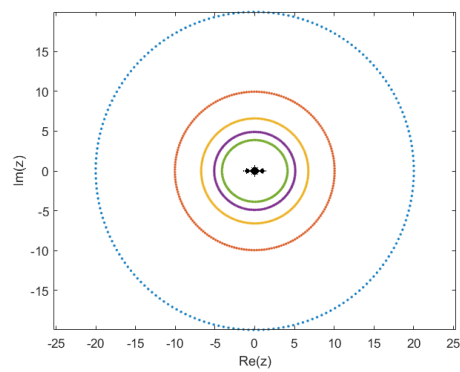
Donat que $(\tau \circ \varphi)$ conjuga les aplicacions Q_c i z^2 , es veu clarament que podem estudiar el comportament en la conca d'atracció d'infinit si pujem des d'on es troben definides les circumferències en el domini de z^2 cap al domini de Q_c . Això ve donat pel fet que les circumferències més properes a l'origen, en passar per φ i després per τ acaben anant a zones llunyanes a l'origen i l'inrevés per les circumferències de major radi un cop els hi apliquem $\tau \circ \varphi$.

A continuació, visualitzem les imatges d'aquests conjunts sota $\tau \circ \varphi_c$:

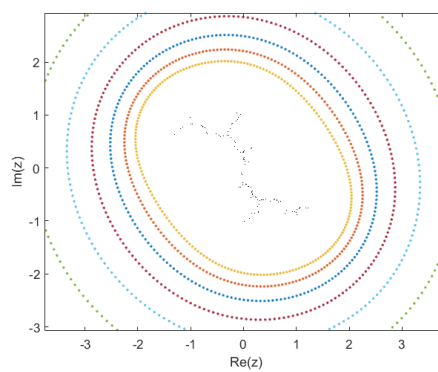
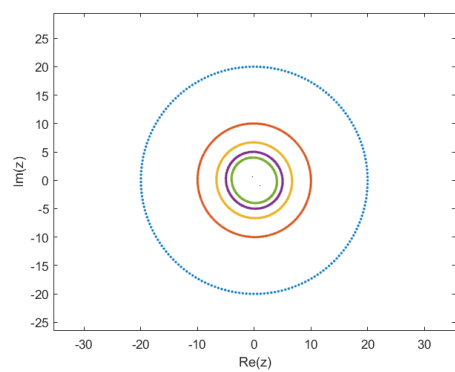
$$c = 0.25 + 0.25i$$

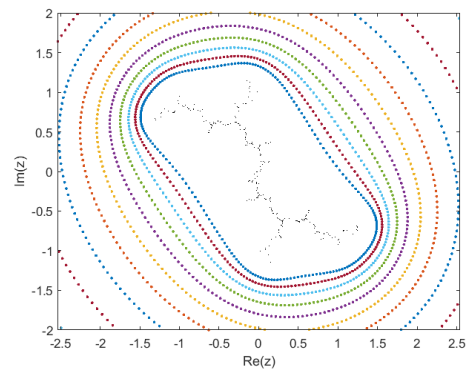


$$c = -1$$

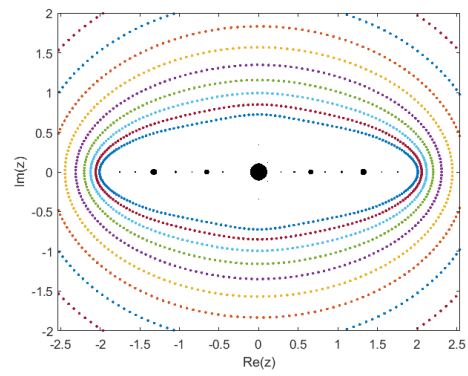
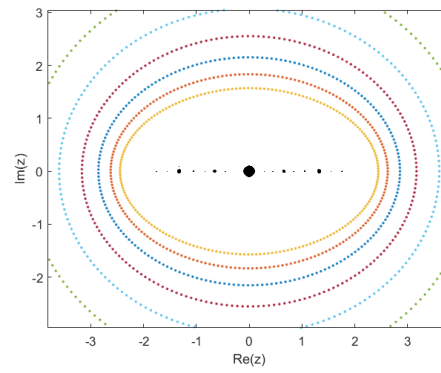
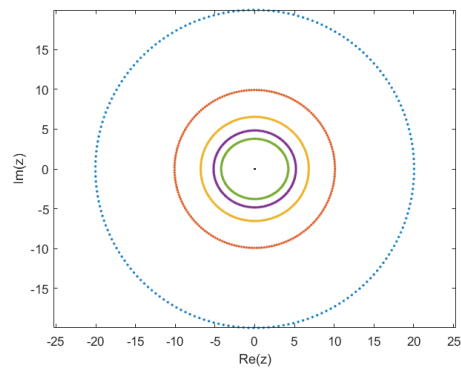


$$c = i$$

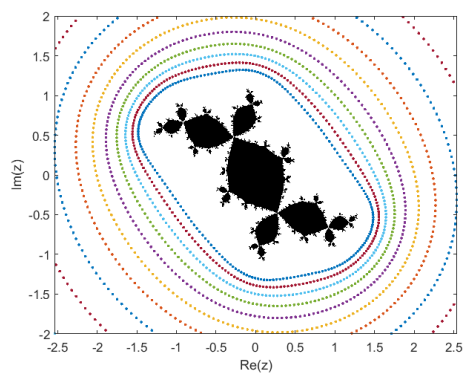
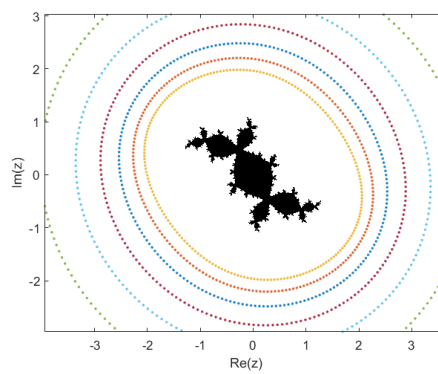
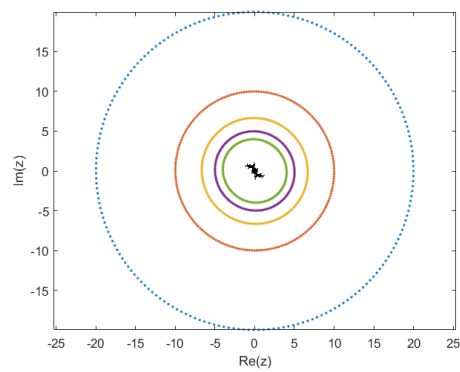




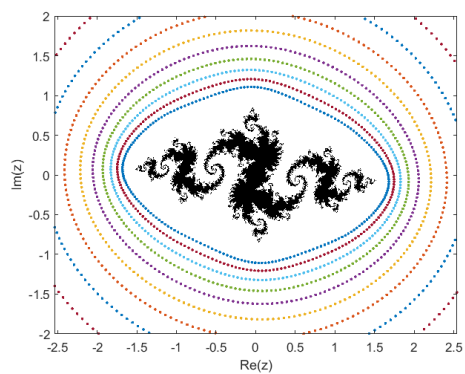
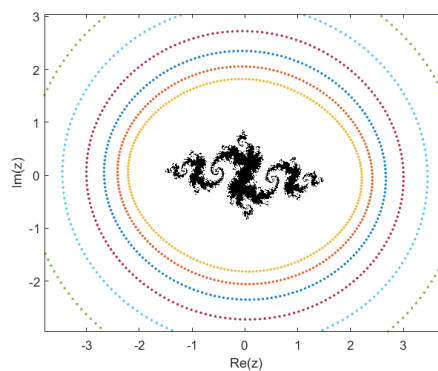
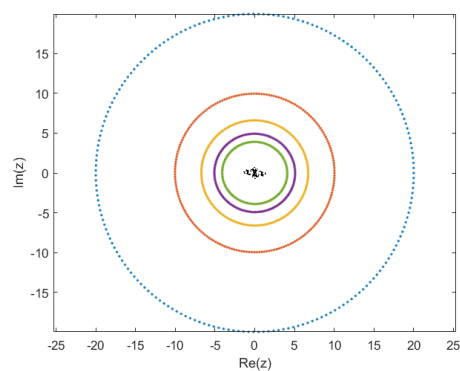
$$c = -1.7549$$



$$c = -0.12256 + 0.74486i$$



$$c = -0.796145 + 0.148172i$$

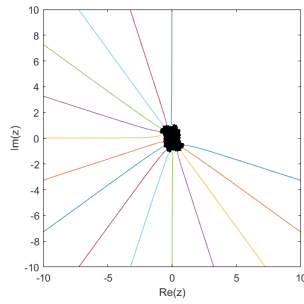


7.2 Línies cap a l'infinit

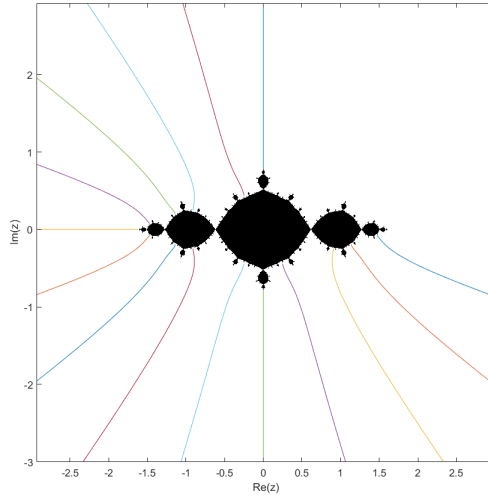
En aquest apartat el que estem visualitzant són les imatges de $(\tau \circ \varphi)$ pel conjunt $\{re^{2\pi i s_k} \mid 0 \leq r \leq 1\}$ on, com es pot veure als annexos, fixem un valor de s_k i representem els productes per les diferents r entre 0 i 1.

Llavors fent això es pot veure com passa una cosa similar a l'apartat anterior, a partir de fixar un angle i anar augmentant el radi passem de punts que es troben molt lluny del conjunt ple de Fatou (a la conca d'atracció de l'infinit de Q_c), a punts propers a la seva frontera, és a dir, al conjunt de Julia. Això no termina de succeir pel cas $c = -1.7549$, aquest fet pot venir donat perquè, com hem vist anteriorment, aquest era el cas en el qual els coeficients de la conjugació φ tenien més error.

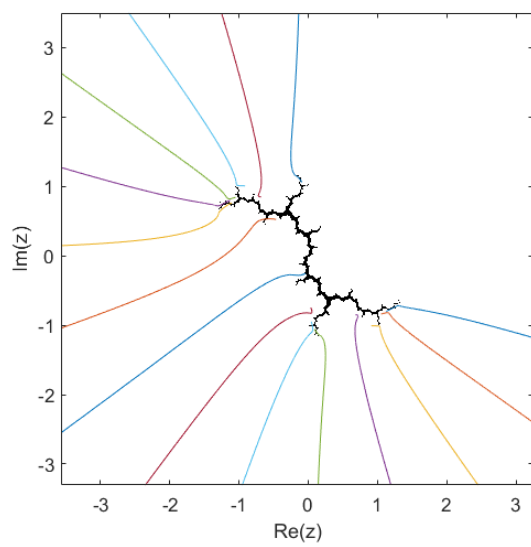
$$c = 0.25 + 0.25i$$



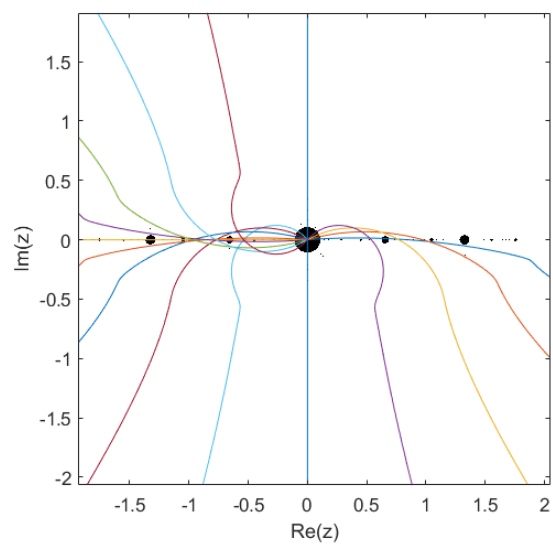
$$c = -1$$



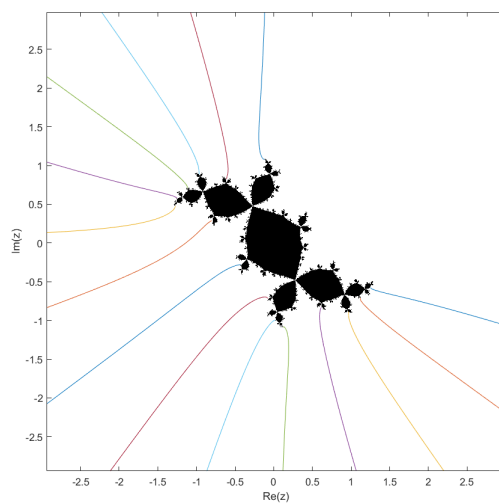
$$c = i$$



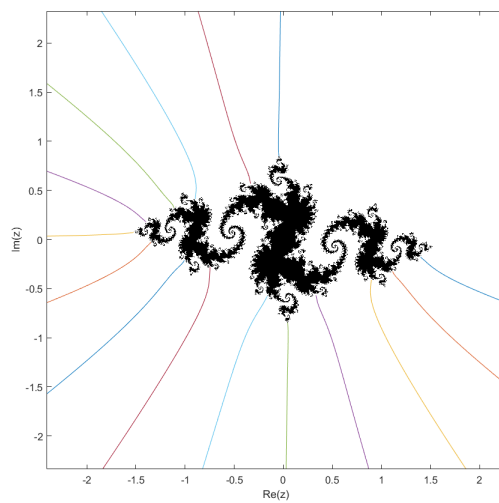
$$c = -1.7549$$



$$c = -0.12256 + 0.74486i$$



$$c = -0.796145 + 0.148172i$$



8 Annex

Codi Conjunt Ple de Fatou

```
1 x=linspace(-2,2,1000);
2 y=linspace(-2,2,1000);
3 [X,Y]=meshgrid(x,y); %X és una matriu on totes les files estan formades pel vector x, i Y és
   s una matriu on totes les columnes estan formades pel vector y
4 Z=X+1i*Y;
5 N=12; %Number of iterations
6
7 %Values of c
8 %c=-0.796145 + 0.148172*1i;
9 %c=0.25+0.25*1i;
10 %c=-0.12256 + 0.74486*1i;
11 %c=-1;
12 %c=1i;
13 c=-1.7549;
14
15 %Polinomi de coeficients i variable complexos
16 Q_c=@(z) z.^2 + c;
17 R=10; %Radi d'escapament
18 Z1=Z;
19
20 %Comprovacio de punts amb orbites acotades
21 BM=ones(size(Z));
22 for i=1:N
23     Z1=Q_c(Z1);
24 end
25
26
27 for i=1:size(Z1,1)
28     for j=1:size(Z1,2)
29         if isnan(abs(Z1(i,j))) || abs(Z1(i,j))>R
30             BM(i,j)=0;
31         end
32     end
33 end
34
35 %Representacio Conjunt Ple de Fatou
36 figure;
37 imagesc(x, y, BM);
38 colormap([0 0 0; 1 1 1]);
39 axis xy;
40 xlabel('Re(z)'); ylabel('Im(z)');
41 realPart = real(c);
42 imagPart = imag(c);
43 titleStr = sprintf('K(Q_c)_{per_c}=%.6f+%.6fi', realPart, imagPart);
44 title(titleStr, 'Interpreter', 'latex')
```

Codi Conjunt de Julia

```
1 %Valors del Prametere c
2 %c=-0.796145 + 0.148172*1i;
3 %c=-0.12256 + 0.74486*1i;
4 %c=-1;
5 c=0.25+0.25*1i;
6 %c=1i;
7 %c=-1.7549;
8
9 %Funció i Derivada
10 Q_c=@(z) z.^2+c;
11 dQ_c=@(z) 2.*z;
12
13 %Punts Fixos i Comprovació punt fix repulsor
14 z1=(1+sqrt(1-4*c))/2;
15 z2=(1-sqrt(1-4*c))/2;
16 abs1=abs(dQ_c(z1));
17 abs2=abs(dQ_c(z2));
18 if abs1 > 1
19     preimatges=z1;
20 else
21     preimatges=z2;
22 end
23
24 N=10; % Quantitat de iteracions
25 for i=1:N
26     newpre=[];
27     for j=1:length(preimatges)
28         w = preimatges(j);
29         % Les preimatges son les solucions de z^2-w+c=0
30         a = 1;
31         b = 0;
32         d = c-w;
33         arrel2 = b^2 - 4*a*d;
34         x1 = (-b + sqrt(arrel2)) / (2*a);
35         x2 = (-b - sqrt(arrel2)) / (2*a);
36         newpre = [newpre, x1, x2];
37     end
38     % Afegim les noves preimatges al vector de preimatges
39     preimatges = [preimatges, newpre];
40 end
41
42 % Visualització conjunt de Julia
43 figure;
44 plot(real(preimatges), imag(preimatges), '.', 'MarkerSize', 5);
45 axis equal;
46 title('Preimatges del punt fix repulsor');
47 xlabel('Re');
48 ylabel('Im');
```

Codi Càlcul Coeficients Conjugació

```
1
2 N_max=100; %Longitud vectors
3 N_max2=100; %Longitud de la serie de Taylor
4 it=0:1:N_max; %Vector d'iteracions
5 syms z
6
7 %Polinomi de coeficients i variable complexos
8 %c=-0.796145 + 0.148172*1i;
9 %c=-0.12256 + 0.74486*1i;
10 c=0.25+0.25*1i;
11 %c=-1;
12 %c=1i;
13 %c=-1.7549;
14 f=z^2/(1+z^2*c);
15
16 coef_an = zeros(1, N_max + 1);
17 b_coef = zeros(1, N_max+1);
18
19 %Condicions donades
20 b_coef(1)=0;
21
22 %Coeficients an
23 for i = 0:N_max2
24     an = diff(f, z, i);
25     coef_an(i+1) = double(subs(an, z, 0)) / factorial(i);
26 end
27
28 b_coef(2)=1/coef_an(3);
29
30 coef_an_new=coef_an;
31 coef_an_new(1)=0;
32 coef_an_new(2)=0;
33 coef_an_new(3)=0;
34
35 %Trobem els coeficients de la conjugació
36 for i=3:N_max
37     j=i-1;
38     if mod(i,2)==1 %Cas on i és senar
39         k=j/2;
40         comp=composicio2(coef_an_new,b_coef,2*k+1);
41         H=comp(2*k+2);
42         l=1:k-1;
43         suma=sum(b_coef(l+2).*b_coef(2*k-l+1));
44         b_coef(2*k+1)=-(H+coef_an(2)*suma)/2;
45     else %Cas on i és parell
46         k=(j+1)/2;
47         comp=composicio2(coef_an_new,b_coef,2*k);
48         H=comp(2*k+1);
49         l=2:k-1;
```



```

50         suma=sum(b_coef(1+1).*b_coef(2*k-1+1));
51         b_coef(2*k)=(b_coef(k+1)-H-2*coef_an(3)*suma-coef_an(3)*b_coef(k+1)^2)/(2);
52     end
53 end
54
55 %Comprovació
56 z_square=zeros(1,N_max+1);
57 z_square(3)=1;
58 Comprovacio1=composicio2(coef_an,b_coef,N_max); % f o phi
59 Comprovacio2=composicio2(b_coef,z_square,N_max); % phi o z^2
60 dif=abs(resta_activitat1(Comprovacio1, Comprovacio2));
61
62 %Grafic de comprovacions f(phi(x)) i phi(f'(0)*x)
63 figure;
64 plot(it, abs(Comprovacio1), 'b-o', 'LineWidth', 1.5, 'MarkerSize', 5);
65 hold on;
66 plot(it, abs(Comprovacio2), 'r-s', 'LineWidth', 1.5, 'MarkerSize', 5);
67 hold off;
68 xlabel('Iteraciones_(it)');
69 ylabel('Valors_de_comprovació');
70 legend('Comprovacio1_f(phi(z))', 'Comprovacio2_phi(z^2)', 'Location', 'best');
71 title('Comparació_de_comprovacions');
72 grid on;
73
74 %Grafic d'evolucio de l'error
75 figure;
76 plot(it, dif,'g-d', 'LineWidth', 1.5, 'MarkerSize', 5);
77 xlabel('Iteraciones_(it)');
78 ylabel('Diferencias_|f(phi(z))_phi(f''(z^2))|');
79 title('Diferencia_entre_comprovacions');
80 grid on;

```

Codis Imatges de $\tau \circ \varphi$

Conjunt $\{s_k e^{2\pi i \theta} \mid 0 \leq \theta \leq 1\}$

```
1 N_max=100; %Longitud vectors
2 N_max2=100; %Longitud de la srie de Taylor
3 it=0:1:N_max; %Vector d'iteracions
4 syms z
5
6 %Polinomi de coeficients i variable complexos
7 %c=-0.796145 + 0.148172*1i;
8 c=-0.12256 + 0.74486*1i;
9 %c=0.25+0.25i;
10 %c=-1;
11 %c=1i;
12 %c=-1.7549;
13 f=z^2/(1+z^2*c);
14
15 coef_an = zeros(1, N_max + 1);
16 b_coef = zeros(1, N_max+1);
17
18 %Condicions donades
19 b_coef(1)=0;
20
21 %Coeficients an
22 for i = 0:N_max2
23     an = diff(f, z, i);
24     coef_an(i+1) = double(subs(an, z, 0)) / factorial(i);
25 end
26
27 b_coef(2)=1/coef_an(3);
28
29 %Creem un nou vector de coef_an ja que el sumatori de les H comena en coef_an(4)
30 coef_an_new=coef_an;
31 coef_an_new(1)=0;
32 coef_an_new(2)=0;
33 coef_an_new(3)=0;
34
35 %Trobem els coeficients de la conjugació
36 for i=3:N_max
37     j=i-1;
38     if mod(i,2)==1 %Cas senar
39         k=j/2;
40         comp=composicio2(coef_an_new,b_coef,2*k+1);
41         H=comp(2*k+2);
42         l=1:k-1;
43         suma=sum(b_coef(l+2).*b_coef(2*k-l+1));
44         b_coef(2*k+1)=-(H+coef_an(2)*suma)/2;
45     else %Cas parell
46         k=(j+1)/2;
47         comp=composicio2(coef_an_new,b_coef,2*k);
```

```

48     H=comp(2*k+1);
49     l=2:k-1;
50     suma=sum(b_coef(l+1).*b_coef(2*k-l+1));
51     b_coef(2*k)=(b_coef(k+1)-H-2*coef_an(3)*suma-coef_an(3)*b_coef(k+1)^2)/(2);
52     end
53 end
54
55 %Definim el conjunt s_k
56 radis=zeros(1,15);
57 for i=1:15
58     radis(i)=0.05*i;
59 end
60
61 PHI = @(z) sum(b_coef .* z.^(0:length(b_coef)-1));
62 alpha=linspace(0,1,200); %Vector de angles entre 0 i 1
63 circ1=zeros(1,200); %Vector per emmagatzemar punts circumferncia
64 images1=zeros(1,200); %Vector per emmagatzemar les imatges de la composició de tau i phi
65
66
67 %Representació Conjunt de Julia i de Fatou
68 x=linspace(-2,2,1000);
69 y=linspace(-2,2,1000);
70 [X,Y]=meshgrid(x,y); %X és una matriu on totes les files estan formades pel vector x, i Y és
    s una matriu on totes les columnes estan formades pel vector y
71 Z=X+1i*Y;
72 N=100; %Number of iterations
73
74 %Polynomial of complex coefficients and variable
75 Q_c=@(z) z.^2 + c;
76 R=10; %Radi d'escapament
77 Z1=Z;
78
79 %Representació conjunt ple de Fatou
80 BM=ones(size(Z));
81 for i=1:N
82     Z1=Q_c(Z1);
83 end
84
85
86 for i=1:size(Z1,1)
87     for j=1:size(Z1,2)
88         if isnan(abs(Z1(i,j))) || abs(Z1(i,j))>R
89             BM(i,j)=0;
90         end
91     end
92 end
93
94 figure;
95 imagesc(x, y, BM);
96 colormap([1 1 1; 0 0 0]);

```

```

97 axis xy;
98 axis equal;
99 xlabel('Re(z)');
100 ylabel('Im(z)');
101 hold on;
102 colors = lines(5); % Crea una paleta de 5 colors diferents
103 for r = 1:15
104     for i=1:length(circ1)
105         circ1(i)=radis(r)*exp(2*pi*1j*alpha(i));
106     end
107
108     for i=1:length(images1)
109         images1(i)=1/PHI(circ1(i));
110     end
111     plot(real(images1), imag(images1), '.', 'MarkerSize', 5);
112     hold on;
113 end

```

Conjunt $\{re^{2\pi i s_k} \mid 0 \leq r \leq 1\}$

```

1 N_max=100; %Longitud vectors
2 N_max2=100; %Longitud de la srie de Taylor
3 it=0:1:N_max; %Vector d'iteracions
4 syms z
5
6 %Polinomi de coeficients i variable complexos
7 %c=-0.796145 + 0.148172*1i;
8 %c=-0.12256 + 0.74486*1i;
9 %c=0.25+0.25i;
10 %c=-1;
11 %c=1i;
12 c=-1.7549;
13 f=z^2/(1+z^2*c);
14
15 coef_an = zeros(1, N_max + 1);
16 b_coef = zeros(1, N_max+1);
17
18 %Condicions donades
19 b_coef(1)=0;
20
21 %Coeficients an
22 for i = 0:N_max2
23     an = diff(f, z, i);
24     coef_an(i+1) = double(subs(an, z, 0)) / factorial(i);
25 end
26
27 b_coef(2)=1/coef_an(3);
28

```

```

29 %Creem un nou vector de coef_an ja que el sumatori de les H comena en coef_an(4)
30 coef_an_new=coef_an;
31 coef_an_new(1)=0;
32 coef_an_new(2)=0;
33 coef_an_new(3)=0;
34
35 %Obtenim els coeficients de la conjugació
36 for i=3:N_max
37     j=i-1;
38     if mod(i,2)==1 %Cas Senar
39         k=j/2;
40         comp=composicio2(coef_an_new,b_coef,2*k+1);
41         H=comp(2*k+2);
42         l=1:k-1;
43         suma=sum(b_coef(l+2).*b_coef(2*k-l+1));
44         b_coef(2*k+1)=-(H+coef_an(2)*suma)/2;
45     else %Cas Parell
46         k=(j+1)/2;
47         comp=composicio2(coef_an_new,b_coef,2*k);
48         H=comp(2*k+1);
49         l=2:k-1;
50         suma=sum(b_coef(l+1).*b_coef(2*k-l+1));
51         b_coef(2*k)=(b_coef(k+1)-H-2*coef_an(3)*suma-coef_an(3)*b_coef(k+1)^2)/(2);
52     end
53 end
54
55 %Definim el conjunt s_k
56 radis=zeros(1,15);
57 for i=1:15
58     radis(i)=0.05*i;
59 end
60
61 PHI = @(z) sum(b_coef .* z.^(0:length(b_coef)-1));
62 alpha=linspace(0,1,1000); %En aquest cas representa els radis entre 0 i 1
63 circ1=zeros(1,1000); %Emmagatzema les imatges de r*(2*pi*i*s_k)
64 images1=zeros(1,1000); %Emmagatzema les imatges de tau compostat amb phi
65
66
67 %Representació Conjunt de Julia i de Fatou
68 x=linspace(-2,2,2000);
69 y=linspace(-2,2,2000);
70 [X,Y]=meshgrid(x,y); %X és una matriu on totes les files estan formades pel vector x, i Y és
    %una matriu on totes les columnes estan formades pel vector y
71 Z=X+1i*Y;
72 N=100; %Number of iterations
73
74 %Polynomial of complex coefficients and variable
75 Q_c=@(z) z.^2 + c;
76 R=10; %Radi d'escapament
77 Z1=Z;

```

```

78
79 %Versio 1
80
81 BM=ones(size(Z));
82 for i=1:N
83     Z1=Q_c(Z1);
84 end
85
86
87 for i=1:size(Z1,1)
88     for j=1:size(Z1,2)
89         if isnan(abs(Z1(i,j))) || abs(Z1(i,j))>R
90             BM(i,j)=0;
91         end
92     end
93 end
94
95 figure;
96 imagesc(x, y, BM);
97 colormap([1 1 1; 0 0 0]);
98 axis xy;
99 axis equal;
100 xlabel('Re(z)');
101 ylabel('Im(z)');
102 hold on;
103 colors = lines(5); % Crea una paleta de 5 colors diferents
104 for r = 1:15
105     for i=1:length(circ1)
106         circ1(i)=alpha(i)*exp(2*pi*1j*radis(r));
107     end
108
109     for i=1:length(images1)
110         images1(i)=1/PHI(circ1(i));
111     end
112     plot(real(images1), imag(images1), '-', 'MarkerSize', 5);
113     hold on;
114 end
115 xlim([-10,10]);
116 ylim([-10,10]);

```